Project Overview

DevOps CI/CD Platform with Automated Infrastructure and Containerized Deployment

Designed and implemented a complete DevOps CI/CD platform using AWS, Terraform, Jenkins, Docker, and Nginx. The infrastructure is provisioned using Terraform which creates a VPC, subnet, security groups, and two EC2 instances for Jenkins master and agent. The Jenkins pipeline is triggered automatically via GitHub webhook when code is pushed to the repository. The pipeline runs on the Jenkins agent, builds Docker images, and deploys the application using Docker Compose. The application stack consists of three containers: MySQL database, Spring Boot application, and Nginx reverse proxy. Nginx exposes the application to the internet while forwarding requests to the Spring Boot container. This architecture demonstrates infrastructure as code, CI/CD automation, containerization, and cloud-based deployment.

Build a production-style DevOps pipeline that demonstrates the complete lifecycle of modern application delivery, including:
Infrastructure provisioning
Application build
Containerization
Continuous integration
Continuous deployment
Reverse proxy exposure
Database integration
All components are automated and reproducible.

High-Level Architecture

Developer
│
▼
GitHub Repository
│
▼
GitHub Webhook
│
▼
Jenkins Master (EC2)
│
SSH
▼
Jenkins Agent (EC2)
│
Docker Compose
▼

Nginx Reverse Proxy
│
▼
Spring Boot Application
│
▼
MySQL DB

Infrastructure is created using Terraform.
Infrastructure Automation
Resources created:
VPC
Subnet
Internet Gateway
Route Table
Security Groups
Jenkins Master EC2
Jenkins Agent EC2
Elastic IPs

Continuous Integration-When a developer pushes code to GitHub:
git push origin master
GitHub triggers a webhook which calls: http://JENKINS_MASTER:8080/github-webhook/
This automatically starts the Jenkins pipeline.

Continuous Deployment the Jenkins pipeline automatically:
Pulls source code
Builds Docker images
Creates containers
Deploys the application
Containerized Application

The application is deployed as three containers:
MySQL-Database
Spring-Boot Application
Nginx-Reverse proxy

Application Exposure
The application is exposed via Nginx on: http://AGENT_IP:81

System Configuration:
AWS:

```
devops-rohit@enveu-ash-RedmiBook-14:~$ aws --version
aws-cli/2.33.15 Python/3.13.11 Linux/6.14.0-37-generic exe/x86_64.ubuntu.24
devops-rohit@enveu-ash-RedmiBook-14:~$ aws configure
AWS Access Key ID [****************LGVZ]:
AWS Secret Access Key [****************8Lep]:
Default region name [ap-south-1]:
Default output format [json]:
```

GH-CLI:

```
devops-rohit@enveu-ash-RedmiBook-14:~$ cd devops-aws-jenkins-docker-cicd-platform/
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ gh --version
gh version 2.45.0 (2025-07-18 Ubuntu 2.45.0-1ubuntu0.3)
https://github.com/cli/cli/releases/tag/v2.45.0
```

GIT:

```
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ git --version
git version 2.43.0
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ git init
Reinitialized existing Git repository in /home/devops-rohit/devops-aws-jenkins-docker-cicd-
platform/.git/
```

TERRAFORM:

```
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ cd terraform/
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-
platform/terraform$ terraform --version
Terraform v1.14.6
on linux_amd64
+ provider registry.terraform.io/hashicorp/aws v6.35.1
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-
platform/terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.35.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
```

rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

DIRECTORY AND FILE STRUCTURE:

```
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform/terraform$ cd ..
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ tree
.
├── database
│   └── init.sql
├── docker
│   ├── docker-compose.yml
│   └── Dockerfile
├── jenkins
│   ├── agent-conf.sh
│   ├── Jenkinsfile
│   └── master-conf.sh
├── nginx
│   ├── sites-available
│   │   └── nw-nginx.conf
│   └── sites-enabled
├── pom.xml
├── README.md
├── src
│   └── main
│   ├── java
│   │   └── com
│   │   └── rohit
│   │   └── portfolio
│   │   ├── controller
│   │   │   ├── HomeController.java
│   │   │   └── ProjectController.java
│   │   ├── model
│   │   │   ├── Profile.java
│   │   │   ├── Project.java
│   │   │   └── Skill.java
│   │   ├── PortfolioApplication.java
│   │   ├── repository
│   │   │   ├── ProfileRepository.java
│   │   │   ├── ProjectRepository.java
│   │   │   └── SkillRepository.java
│   │   └── service
```

```
│   │   └── PortfolioService.java
│   └── resources
│       ├── application.properties
│       ├── static
│       │   └── css
│       │       └── style.css
│       └── templates
│           ├── index.html
│           └── projects.html
└── terraform
    ├── ec2.tf
    ├── elasticip.tf
    ├── internetgateway.tf
    ├── outputs.tf
    ├── provider.tf
    ├── securitygroup.tf
    ├── subnet.tf
    ├── variables.tf
    └── vpc.tf

22 directories, 32 files
```

devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ cat database/init.sql docker/docker-compose.yml docker/Dockerfile jenkins/agent-conf.sh jenkins/Jenkinsfile jenkins/master-conf.sh nginx/sites-available/nw-nginx.conf pom.xml README.md src/main/java/com/rohit/portfolio/controller/HomeController.java src/main/java/com/rohit/portfolio/controller/ProjectController.java src/main/java/com/rohit/portfolio/model/Profile.java src/main/java/com/rohit/portfolio/model/Project.java src/main/java/com/rohit/portfolio/model/Skill.java src/main/java/com/rohit/portfolio/PortfolioApplication.java src/main/java/com/rohit/portfolio/repository/ProfileRepository.java src/main/java/com/rohit/portfolio/repository/ProjectRepository.java src/main/java/com/rohit/portfolio/repository/SkillRepository.java src/main/java/com/rohit/portfolio/service/PortfolioService.java src/main/resources/application.properties src/main/resources/static/css/style.css src/main/resources/templates/index.html src/main/resources/templates/projects.html terraform/ec2.tf terraform/elasticip.tf terraform/internetgateway.tf terraform/outputs.tf terraform/securitygroup.tf terraform/subnet.tf terraform/variables.tf terraform/vpc.tf

INIT.SQL:

```
CREATE DATABASE IF NOT EXISTS portfolio;
```

```
USE portfolio;

-- ================================
-- Profile Table
-- ================================

CREATE TABLE IF NOT EXISTS profile(

id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100) NOT NULL,
title VARCHAR(200),
summary TEXT,
email VARCHAR(100),
linkedin VARCHAR(200)

);

-- ================================
-- Skill Table
-- ================================

CREATE TABLE IF NOT EXISTS skill(

id INT PRIMARY KEY AUTO_INCREMENT,
skill_name VARCHAR(100) NOT NULL

);

-- ================================
-- Project Table
-- ================================

CREATE TABLE IF NOT EXISTS project(

id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(200) NOT NULL,
github_url VARCHAR(500),
description TEXT,
technology VARCHAR(200)

);
```

```
-- ================================
-- Insert Profile
-- ================================

INSERT INTO profile
(name,title,summary,email,linkedin)

VALUES
('Rohit Rawat',
'DevOps Engineer | AWS | Docker | Jenkins | Terraform',
'DevOps engineer experienced in CI/CD, containerization, AWS infrastructure and automation.',
'rawatrohit.wrk.25@gmail.com',
'https://linkedin.com/in/rohit-rawat');

-- ================================
-- Insert Skills
-- ================================

INSERT INTO skill(skill_name) VALUES
('AWS'),
('Docker'),
('Jenkins'),
('Terraform'),
('Linux'),
('Nginx'),
('Git');

-- ================================
-- Insert Projects
-- ================================

INSERT INTO project(name,github_url,description,technology)

VALUES
(
'Jenkins Master Agent CI/CD',
'https://github.com/rohitrawat025/devops-aws-jenkins-docker-cicd-platform',
'Implemented Jenkins distributed pipeline on AWS EC2 with automated Docker deployment',
'Jenkins, Docker, AWS'
),
```

```
(
'Spring Boot Docker Deployment',
'https://github.com/rohitrawat025/springboot-docker-deployment',
'Containerized Java application with Docker and Nginx reverse proxy',
'Docker, Nginx, Spring Boot'
),

(
'Terraform AWS Infrastructure',
'https://github.com/rohitrawat025/aws-terraform-infrastructure',
'Provisioned VPC, EC2, IAM and networking infrastructure using Terraform',
'AWS, Terraform'
);
```

DOCKER-COMPOSE:

```
version: "3.8"

services:

 mysql:
 image: mysql:8
 container_name: portfolio-mysql
 restart: always
 environment:
 MYSQL_ROOT_PASSWORD: root
 MYSQL_DATABASE: portfolio
 volumes:
 - mysql-data:/var/lib/mysql
 - ../database/init.sql:/docker-entrypoint-initdb.d/init.sql
 networks:
 - portfolio-net

 springboot:
 build:
 context: ..
 dockerfile: docker/Dockerfile
 container_name: portfolio-app
 restart: always
 depends_on:
 - mysql
 environment:
 SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/portfolio
```

```
 SPRING_DATASOURCE_USERNAME: root
 SPRING_DATASOURCE_PASSWORD: root
 networks:
 - portfolio-net

 nginx:
 image: nginx:latest
 container_name: portfolio-nginx
 restart: always
 depends_on:
 - springboot
 ports:
 - "81:81"
 volumes:
 - ../nginx/sites-enabled:/etc/nginx/conf.d
 networks:
 - portfolio-net

volumes:
 mysql-data:

networks:
 portfolio-net:
```

DOCKERFILE:

```
FROM maven:3.9.9-eclipse-temurin-17 AS builder

WORKDIR /app

COPY pom.xml .
RUN mvn -B -q -e -DskipTests dependency:go-offline

COPY src ./src

RUN mvn clean package -DskipTests


FROM eclipse-temurin:17-jre-jammy

RUN useradd -ms /bin/bash spring

WORKDIR /app
```

```
COPY --from=builder /app/target/*.jar app.jar

RUN chown spring:spring app.jar

USER spring

EXPOSE 8080

ENTRYPOINT ["java","-jar","/app/app.jar"]
```

AGENT EC2 CONFIGURE:

```
#!/bin/bash
set -e

sudo apt update -y

# Added 'ca-certificates' and 'gnupg' for repository management
sudo apt install -y git curl wget unzip software-properties-common ca-certificates gnupg

# Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources
echo \
 "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu \
 $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
 sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update -y

sudo apt install -y openjdk-17-jdk

# Install the engine and the plugin
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl start docker
sudo systemctl enable docker

# Allow 'ubuntu' user to run docker commands without sudo
```

```
sudo usermod -aG docker ubuntu

# Verifying
git --version
docker --version
docker compose version
java --version

echo "Agent setup complete"
```

JENKINSFILE:

```
pipeline {

 agent { label 'rohit-agent-1' }

 triggers {
 githubPush()
 }

 environment {
 COMPOSE_FILE = "docker/docker-compose.yml"
 }

 stages {

 stage('Clean Workspace') {
 steps {
 deleteDir()
 }
 }

 stage('Checkout Code') {
 steps {
 checkout scm
 }
 }

 stage('Verify Repository') {
 steps {
 sh '''
 echo "Repository structure:"
 ls -la
```

```
'''
}
}

stage('Prepare Nginx Config') {
steps {
sh '''
mkdir -p nginx/sites-enabled
cp nginx/sites-available/nw-nginx.conf nginx/sites-enabled/
'''
}
}

stage('Full Docker Cleanup') {
steps {
sh '''
echo "Stopping containers..."

docker compose -f $COMPOSE_FILE down -v || true

echo "Removing unused images..."
docker system prune -af || true
'''
}
}

stage('Build Docker Images') {
steps {
sh '''
docker compose -f $COMPOSE_FILE build
'''
}
}

stage('Deploy Containers') {
steps {
sh '''
docker compose -f $COMPOSE_FILE up -d
'''
}
}
```

```
stage('Wait For Application') {
steps {
sh '''
echo "Waiting for application..."

for i in {1..20}
do
if curl -s http://localhost:81 > /dev/null; then
echo "Application is ready"
exit 0
fi

echo "Waiting..."
sleep 5
done

echo "Application failed to start"
exit 1
'''
}
}

}

post {

success {
sh '''
echo "Application URL:"
curl -s ifconfig.me
'''
}

failure {
sh '''
echo "Deployment failed"

docker ps -a
docker logs portfolio-app || true
'''
}
```

```
 }

}
```

MASTER EC2 CONFIGURE:

```
#!/bin/bash

# Jenkins Installation Script for Ubuntu Jammy/Noble
# This script fixes GPG key issues, installs Java 17, and Jenkins.



set -e # Exit on any error

# Install dirmngr for keyserver communication
sudo apt update
sudo apt install dirmngr -y

# Setup GPG directory for root
sudo mkdir -p /root/.gnupg
sudo chmod 700 /root/.gnupg

# Import Jenkins GPG Key
sudo gpg --no-default-keyring \
--keyring /usr/share/keyrings/jenkins-keyring.gpg \
--keyserver keyserver.ubuntu.com \
--recv-keys 7198F4B714ABFC68

# Key Permissions
sudo chmod 644 /usr/share/keyrings/jenkins-keyring.gpg

# Add Jenkins Repository
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.gpg] https://pkg.jenkins.io/debian-stable
binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

# Install Java
sudo apt update
sudo apt install openjdk-17-jdk -y

# Install Jenkins
sudo apt install jenkins -y
sudo systemctl start jenkins
```

```
sudo systemctl enable jenkins
sudo systemctl status jenkins

#verufy
java --version
jenkins --version
# initial password to be generated
echo " ⏳ Waiting for Jenkins to initialize (this may take 30-60 seconds)..."
while [ ! -f /var/lib/jenkins/secrets/initialAdminPassword ]; do
 sleep 2
done
# Admin Password
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
echo "Master setup complete"
```

NW-NGNX.CONF:

```
server {

 listen 81;

 server_name _;

 location / {

 proxy_pass http://springboot:8080;

 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;

 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
 }
}
```

POM.XML:

```
proxy_pass http://springboot:8080;

 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;

 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
```

```
 }
}
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 http://maven.apache.org/xsd/maven-4.0.0.xsd">

 <modelVersion>4.0.0</modelVersion>

 <groupId>com.rohit</groupId>
 <artifactId>portfolio</artifactId>
 <version>1.0</version>
 <name>portfolio</name>
 <description>DevOps Portfolio Application</description>

 <parent>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-parent</artifactId>
 <version>3.3.0</version>
 <relativePath/>
 </parent>

 <properties>
 <java.version>17</java.version>
 </properties>

 <dependencies>

 <!-- Spring Boot Web -->
 <dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-web</artifactId>
 </dependency>

 <!-- Thymeleaf -->
 <dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-thymeleaf</artifactId>
 </dependency>

 <!-- JPA -->
 <dependency>
```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL Driver -->
<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<scope>runtime</scope>
</dependency>

<!-- Lombok -->
<dependency>
<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
<optional>true</optional>
</dependency>

<!-- Test -->
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

README.MD:

DevOps CI/CD Portfolio Platform

A production-style DevOps project demonstrating automated infrastructure provisioning, containerized application deployment, and CI/CD pipeline implementation using AWS, Terraform, Jenkins, Docker, and Nginx.

Architecture

Developer → GitHub → Jenkins Master → Jenkins Agent → Docker Compose → Nginx → Spring Boot → MySQL

Technologies Used

* AWS EC2
* Terraform
* Jenkins (Master–Agent architecture)
* Docker
* Docker Compose
* Nginx Reverse Proxy
* Spring Boot
* MySQL
* GitHub Webhooks
* Linux

Infrastructure Provisioning

Terraform automatically provisions:-

* VPC
* Subnet
* Internet Gateway
* Route Tables
* Security Groups
* Jenkins Master EC2
* Jenkins Agent EC2
* Elastic IPs

CI/CD Workflow

1. Developer pushes code to GitHub
2. GitHub webhook triggers Jenkins
3. Jenkins pipeline runs on agent node
4. Docker images are built
5. Containers are deployed using Docker Compose
6. Nginx exposes application to the internet

Container Architecture

The Jenkins agent runs three containers:

* MySQL Database
* Spring Boot Application
* Nginx Reverse Proxy

Application URLs

Frontend

http://SERVER_IP:81

Projects page

http://SERVER_IP:81/projects

Author

Rohit Rawat
DevOps Engineer
AWS | Docker | Docker Compose | Jenkins | Terraform | Linux

APPLICATION FILES:
package com.rohit.portfolio.controller;

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

import com.rohit.portfolio.service.PortfolioService;

@Controller
public class HomeController {

 private final PortfolioService service;

 public HomeController(PortfolioService service) {
 this.service = service;
 }

 @GetMapping("/")
 public String home(Model model) {

 model.addAttribute("profile", service.getProfile());
 model.addAttribute("skills", service.getSkills());
 model.addAttribute("projects", service.getProjects());

 return "index";
 }
}
package com.rohit.portfolio.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

import com.rohit.portfolio.service.PortfolioService;

@Controller
public class ProjectController {

 private final PortfolioService service;

 public ProjectController(PortfolioService service) {
 this.service = service;
 }
```

```
 @GetMapping("/projects")
 public String projects(Model model) {

 model.addAttribute("projects", service.getProjects());

 return "projects";
 }
}
package com.rohit.portfolio.model;

import jakarta.persistence.*;

@Entity
public class Profile {

 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;

 private String name;
 private String title;
 private String summary;
 private String email;
 private String linkedin;

 public Long getId() { return id; }

 public String getName() { return name; }
 public void setName(String name) { this.name = name; }

 public String getTitle() { return title; }
 public void setTitle(String title) { this.title = title; }

 public String getSummary() { return summary; }
 public void setSummary(String summary) { this.summary = summary; }

 public String getEmail() { return email; }
 public void setEmail(String email) { this.email = email; }

 public String getLinkedin() { return linkedin; }
 public void setLinkedin(String linkedin) { this.linkedin = linkedin; }
}
```

```java
package com.rohit.portfolio.model;

import jakarta.persistence.*;

@Entity
@Table(name = "project")
public class Project {

 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;

 private String name;

 @Column(name = "github_url")
 private String githubUrl;

 @Column(length = 2000)
 private String description;

 private String technology;

 public Long getId() { return id; }

 public String getName() { return name; }
 public void setName(String name) { this.name = name; }

 public String getGithubUrl() { return githubUrl; }
 public void setGithubUrl(String githubUrl) { this.githubUrl = githubUrl; }

 public String getDescription() { return description; }
 public void setDescription(String description) { this.description = description; }

 public String getTechnology() { return technology; }
 public void setTechnology(String technology) { this.technology = technology; }
}
```

```java
package com.rohit.portfolio.model;

import jakarta.persistence.*;

@Entity
@Table(name = "skill")
```

```
public class Skill {

 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;

 @Column(name = "skill_name")
 private String skillName;

 public Long getId() { return id; }

 public String getSkillName() { return skillName; }
 public void setSkillName(String skillName) { this.skillName = skillName; }

}
package com.rohit.portfolio;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PortfolioApplication {

 public static void main(String[] args) {
 SpringApplication.run(PortfolioApplication.class, args);
 }

}
package com.rohit.portfolio.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.rohit.portfolio.model.Profile;

public interface ProfileRepository extends JpaRepository<Profile, Long> {
}
package com.rohit.portfolio.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.rohit.portfolio.model.Project;

public interface ProjectRepository extends JpaRepository<Project, Long> {
}
```

```
package com.rohit.portfolio.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.rohit.portfolio.model.Skill;

public interface SkillRepository extends JpaRepository<Skill, Long> {
}
package com.rohit.portfolio.service;

import java.util.List;

import org.springframework.stereotype.Service;

import com.rohit.portfolio.model.Profile;
import com.rohit.portfolio.model.Project;
import com.rohit.portfolio.model.Skill;
import com.rohit.portfolio.repository.ProfileRepository;
import com.rohit.portfolio.repository.ProjectRepository;
import com.rohit.portfolio.repository.SkillRepository;

@Service
public class PortfolioService {

 private final ProfileRepository profileRepo;
 private final ProjectRepository projectRepo;
 private final SkillRepository skillRepo;

 public PortfolioService(ProfileRepository profileRepo,
 ProjectRepository projectRepo,
 SkillRepository skillRepo) {

 this.profileRepo = profileRepo;
 this.projectRepo = projectRepo;
 this.skillRepo = skillRepo;
 }

 public Profile getProfile() {
 return profileRepo.findAll()
 .stream()
 .findFirst()
 .orElse(null);
 }
```

```
 public List<Project> getProjects() {
 return projectRepo.findAll();
 }

 public List<Skill> getSkills() {
 return skillRepo.findAll();
 }
}
# ================================
# SERVER CONFIG
# ================================
server.port=8080

# ================================
# DATABASE CONFIG
# ================================
spring.datasource.url=jdbc:mysql://mysql:3306/portfolio
spring.datasource.username=root
spring.datasource.password=root

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# ================================
# JPA / HIBERNATE
# ================================
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# ================================
# THYMELEAF
# ================================
spring.thymeleaf.cache=false

# ================================
# LOGGING (Optional)
# ================================
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
body{
font-family: Arial, Helvetica, sans-serif;
```

```
margin:0;
padding:0;
background:#f5f5f5;
}


header{
background:#0f172a;
color:white;
padding:20px;
display:flex;
justify-content:space-between;
align-items:center;
}


nav a{
color:white;
margin-left:20px;
text-decoration:none;
font-weight:bold;
}


.hero{
background:#1e293b;
color:white;
text-align:center;
padding:80px;
}


.about{
padding:40px;
text-align:center;
background:white;
}


.skills{
padding:40px;
background:#f1f5f9;
```

```
text-align:center;
}

.skill-container{
display:flex;
justify-content:center;
flex-wrap:wrap;
}

.skill-card{
background:white;
padding:15px;
margin:10px;
border-radius:8px;
box-shadow:0 2px 6px rgba(0,0,0,0.2);
}

.projects-preview{
padding:40px;
background:white;
}

.project-container{
display:flex;
flex-wrap:wrap;
justify-content:center;
}

.project-card{
background:white;
padding:20px;
margin:20px;
width:250px;
border-radius:8px;
box-shadow:0 4px 10px rgba(0,0,0,0.1);
}
```

```
.project-card a{
display:inline-block;
margin-top:10px;
color:#2563eb;
}


footer{
background:#0f172a;
color:white;
text-align:center;
padding:15px;
}
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>

<title>DevOps Portfolio</title>

<link rel="stylesheet" th:href="@{/css/style.css}">

</head>

<body>

<header>

<h1>DevOps Engineer Portfolio</h1>

<nav>
<a href="/">Home</a>
<a href="/projects">Projects</a>
</nav>

</header>


<section class="hero">

<h2 th:text="${profile.name}">Name</h2>
```

```
<p th:text="${profile.title}">DevOps Engineer</p>

</section>


<section class="about">

<h2>About Me</h2>

<p th:text="${profile.summary}">
Passionate DevOps engineer with experience in AWS, CI/CD and automation.
</p>

</section>



<section class="skills">

<h2>Skills</h2>

<div class="skill-container">

<div class="skill-card" th:each="skill : ${skills}">
<p th:text="${skill.skillName}">AWS</p>
</div>

</div>

</section>



<section class="projects-preview">

<h2>Projects</h2>

<div class="project-container">

<div class="project-card" th:each="project : ${projects}">
```

```
<h3 th:text="${project.name}">Project Name</h3>

<p th:text="${project.description}">
Project Description
</p>

<p>
<b>Technology:</b>
<span th:text="${project.technology}">AWS, Docker</span>
</p>

</div>

</div>

</section>


<footer>

<p>© 2026 DevOps Portfolio</p>

</footer>

</body>
</html>
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>

<title>Projects</title>

<link rel="stylesheet" th:href="@{/css/style.css}">

</head>


<body>


<header>
```

```
<h1>My DevOps Projects</h1>

<nav>
<a href="/">Home</a>
<a href="/projects">Projects</a>
</nav>

</header>


<section class="projects-page">

<div class="project-container">

<div class="project-card" th:each="project : ${projects}">

<h2 th:text="${project.name}">Project</h2>

<p th:text="${project.description}">Description</p>

<p>
<b>Technology:</b>
<span th:text="${project.technology}">AWS, Docker</span>
</p>

<a th:href="${project.githubUrl}" target="_blank">Github Repository</a>

</div>

</div>

</section>


<footer>

<p>© 2026 DevOps Portfolio</p>

</footer>
```

```
</body>

</html>
```

TERRAFORM CODE FILES:

EC2S:

```
resource "aws_instance" "jenkins_master" {
 ami = var.ami_id
 instance_type = var.instance_type
 subnet_id = aws_subnet.main_subnet.id
 vpc_security_group_ids = [aws_security_group.main_sg.id]
 key_name = var.key_name

 root_block_device {
 volume_size = 15
 volume_type = "gp3"
 }

 tags = { Name = "jenkins_master" }
}

resource "aws_instance" "jenkins_agent" {
 ami = var.ami_id
 instance_type = var.instance_type
 subnet_id = aws_subnet.main_subnet.id
 vpc_security_group_ids = [aws_security_group.main_sg.id]
 key_name = var.key_name

 root_block_device {
 volume_size = 15
 volume_type = "gp3"
 }

 tags = { Name = "jenkins_agent" }
}
```

ELASTIC IPS:

```
# --- Elastic IP for Jenkins Master ---
resource "aws_eip" "master_eip" {
 domain = "vpc"
 tags = { Name = "jenkins_master_eip" }
}
```

```
resource "aws_eip_association" "master_assoc" {
 instance_id = aws_instance.jenkins_master.id
 allocation_id = aws_eip.master_eip.id
}

# --- Elastic IP for Jenkins Agent ---
resource "aws_eip" "agent_eip" {
 domain = "vpc"
 tags = { Name = "jenkins_agent_eip" }
}

resource "aws_eip_association" "agent_assoc" {
 instance_id = aws_instance.jenkins_agent.id
 allocation_id = aws_eip.agent_eip.id
}
```

INTERNET GATEWAY:

```
resource "aws_internet_gateway" "igw" {
 vpc_id = aws_vpc.main_vpc.id
 tags = { Name = "igw" }
}

resource "aws_route_table" "public_rt" {
 vpc_id = aws_vpc.main_vpc.id
 route {
 cidr_block = "0.0.0.0/0"
 gateway_id = aws_internet_gateway.igw.id
 }
 tags = { Name = "public-rt" }
}

resource "aws_route_table_association" "public_assoc" {
 subnet_id = aws_subnet.main_subnet.id
 route_table_id = aws_route_table.public_rt.id
}
```

OUTPUTS:

```
output "master_ips" {
 value = {
 private_ip = aws_instance.jenkins_master.private_ip
 public_ip = aws_eip.master_eip.public_ip
 elastic_ip = aws_eip.master_eip.public_ip
 }
```

```
}

output "agent_ips" {
 value = {
 private_ip = aws_instance.jenkins_agent.private_ip
 public_ip = aws_eip.agent_eip.public_ip
 elastic_ip = aws_eip.agent_eip.public_ip
 }
}
```

SECURITY GROUP:

```
resource "aws_security_group" "main_sg" {
 name = "main_sg"
 vpc_id = aws_vpc.main_vpc.id

 ingress {
 description = "SSH"
 from_port = 22
 to_port = 22
 protocol = "tcp"
 cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
 description = "HTTP Default"
 from_port = 80
 to_port = 80
 protocol = "tcp"
 cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
 description = "Nginx Custom"
 from_port = 81
 to_port = 81
 protocol = "tcp"
 cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
 description = "Jenkins Master"
 from_port = 8080
 to_port = 8080
```

```
 protocol = "tcp"
 cidr_blocks = ["0.0.0.0/0"]
 }

 ingress {
 description = "MySQL Internal"
 from_port = 3306
 to_port = 3306
 protocol = "tcp"
 cidr_blocks = [var.vpc_cidr]
 }

 egress {
 from_port = 0
 to_port = 0
 protocol = "-1"
 cidr_blocks = ["0.0.0.0/0"]
 }
}
resource "aws_subnet" "main_subnet" {
 vpc_id = aws_vpc.main_vpc.id
 cidr_block = var.subnet_cidr
 map_public_ip_on_launch = true
 availability_zone = "${var.aws_region}a"
 tags = { Name = "main_subnet" }
}
```

VARIABLES:

```
variable "aws_region" {
 default = "ap-south-1"
}

variable "vpc_cidr" {
 default = "10.0.0.0/16"
}

variable "subnet_cidr" {
 default = "10.0.1.0/24"
}

variable "ami_id" {
 description = "Ubuntu 24.04 LTS AMI ID for ap-south-1"
 default = "ami-0dee22c13ea7a9a67"
```

```
}

variable "instance_type" {
 default = "t3.small"
}

variable "key_name" {
 default = "rohit-ec2-key"
}
```

VPC:

```
resource "aws_vpc" "main_vpc" {
 cidr_block = var.vpc_cidr
 tags = { Name = "main_vpc" }
}
```

CREATION OF AWS INFRA WITH TERRAFORM:

```
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform/terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.35.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform/terraform$ terraform fmt
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform/terraform$ terraform validate
Success! The configuration is valid.

devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform/terraform$ terraform plan
Apply complete! Resources: 12 added, 0 changed, 0 destroyed.
```

Outputs:

```
agent_ips = {
 "elastic_ip" = "43.205.193.15"
 "private_ip" = "10.0.1.180"
 "public_ip" = "43.205.193.15"
}
master_ips = {
 "elastic_ip" = "15.207.93.132"
 "private_ip" = "10.0.1.220"
 "public_ip" = "15.207.93.132"
}
```

CONFIGURING THE EC2S

```
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform/terraform$ cd ..
```

COPYING AGENT CONFIGURE FILE TO AGENT EC2:

```
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ scp -i ../rohit-ec2-key.pem jenkins/agent-conf.sh ubuntu@43.205.193.15:/home/ubuntu
agent-conf.sh 100% 1210 24.6KB/s 00:00
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ ssh -i ../rohit-ec2-key.pem ubuntu@43.205.193.15
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1016-aws x86_64)

ubuntu@ip-10-0-1-180:~$ ls
agent-conf.sh
ubuntu@ip-10-0-1-180:~$ chmod +x agent-conf.sh
ubuntu@ip-10-0-1-180:~$ ls
agent-conf.sh
ubuntu@ip-10-0-1-180:~$ ./agent-conf.sh
ubuntu@ip-10-0-1-180:~$ java --version
openjdk 17.0.18 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-124.04.1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-124.04.1, mixed mode, sharing)
ubuntu@ip-10-0-1-180:~$ git --version
git version 2.43.0
ubuntu@ip-10-0-1-180:~$ docker compose version
Docker Compose version v5.1.0
ubuntu@ip-10-0-1-180:~$ docker --version
Docker version 29.3.0, build 5927d80
ubuntu@ip-10-0-1-180:~$ sudo usermod -aG docker ubuntu
ubuntu@ip-10-0-1-180:~$ newgrp docker
ubuntu@ip-10-0-1-180:~$ sudo systemctl status docker
```

```
● docker.service - Docker Application Container Engine
 Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
 Active: active (running) since Wed 2026-03-11 11:59:57 UTC; 6min ago
TriggeredBy: ● docker.socket
 Docs: https://docs.docker.com
 Main PID: 4768 (dockerd)
 Tasks: 9
 Memory: 27.3M (peak: 28.2M)
 CPU: 364ms
 CGroup: /system.slice/docker.service
 └─4768 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

ubuntu@ip-10-0-1-180:~$
```

COPYING MASTER CONFIGURE FILE TO MASTER EC2:

```
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ scp -i ../rohit-ec2-key.pem jenkins/master-conf.sh ubuntu@15.207.93.132:/home/ubuntu
master-conf.sh 100% 1328 32.0KB/s 00:00
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$
devops-rohit@enveu-ash-RedmiBook-14:~/devops-aws-jenkins-docker-cicd-platform$ ssh -i ../rohit-ec2-key.pem ubuntu@15.207.93.132
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1016-aws x86_64)

ubuntu@ip-10-0-1-220:~$ ls
master-conf.sh
ubuntu@ip-10-0-1-220:~$ chmod +x master-conf.sh
ubuntu@ip-10-0-1-220:~$ ls
master-conf.sh
ubuntu@ip-10-0-1-220:~$ ./master-conf.sh
ubuntu@ip-10-0-1-220:~$ java --version
openjdk 17.0.18 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-124.04.1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-124.04.1, mixed mode, sharing)
ubuntu@ip-10-0-1-220:~$ jenkins --version
2.541.2
ubuntu@ip-10-0-1-220:~$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
 Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
 Active: active (running) since Wed 2026-03-11 11:46:31 UTC; 53s ago
 Main PID: 4629 (java)
 Tasks: 51 (limit: 2274)
 Memory: 530.9M (peak: 539.2M)
```

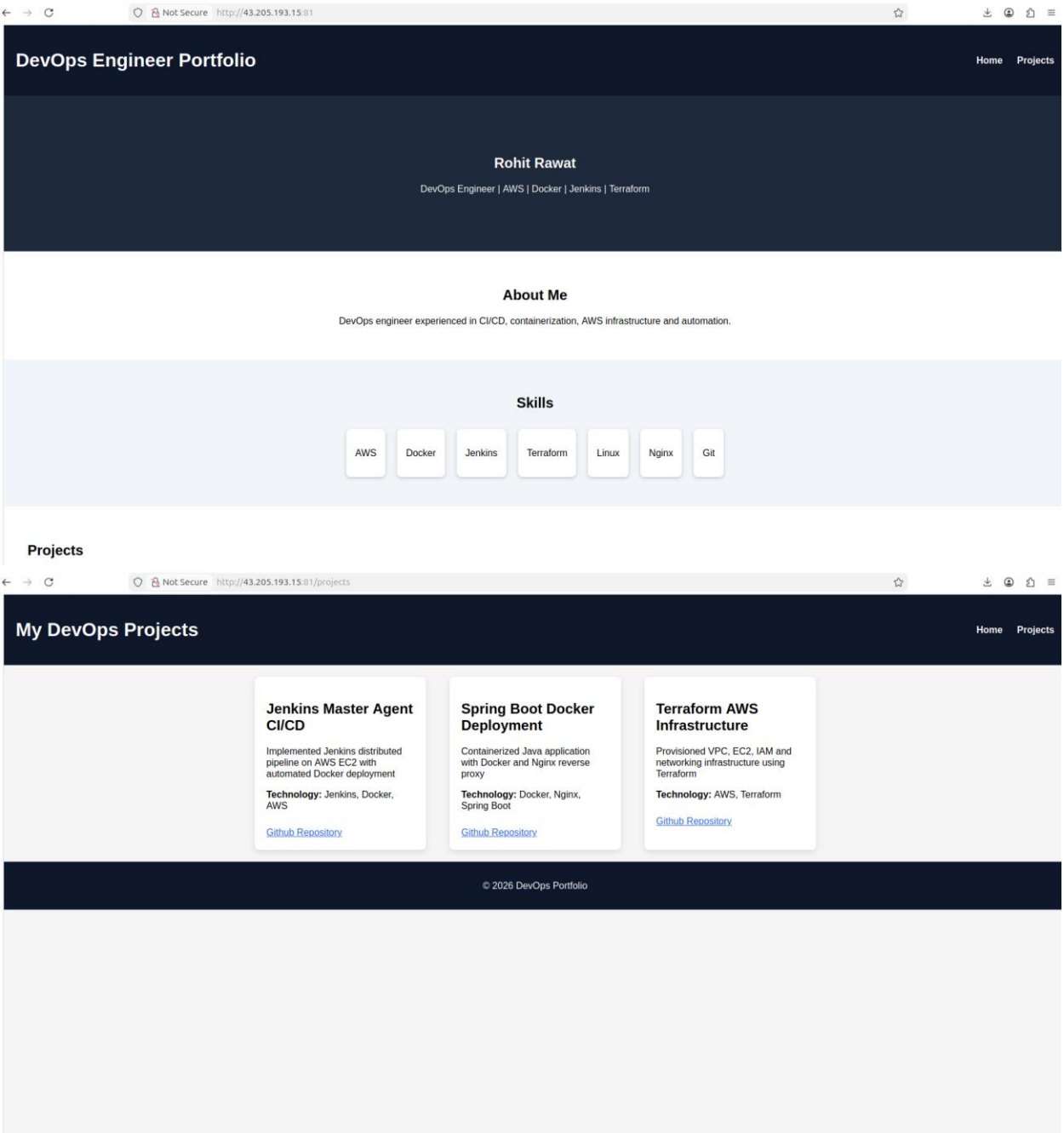
CPU: 24.842s
CGroup: /system.slice/jenkins.service
└─4629 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080
ubuntu@ip-10-0-1-220:/$ sudo cat var/lib/jenkins/secrets/initialAdminPassword

JENKINS UI NODES:
Name: rohit-agent-1
Number of executors: 1
Remote root directory: /home/ubuntu
Labels: rohit-agent-1
Usage: use this node as much as possible
Launch method: launch agent via ssh
Host: 10.0.1.180
Credentials: ubuntu (add-global-ssh username with private key-Scope:Global jenkin, node, item all child item etc-id:ubuntu-username:ubuntu-enter directly: paste privatekey)
Host Key Verification Strategy: non verrifying verification strategy
Availability: keep this agent online as much as possible

JENKINS UI JOB:
New Item: P2 -pipeline-permanent
Discard old builds-Strategy: Log Rotation-Max # of builds to keep: 2
Triggers- GitHub hook trigger for GITScm polling-Pipeline-Definition: peipeline script from scm-SCM: git Repositories-Repository URL: https://github.com/rohitrawat025/devops-aws-jenkins-docker-cicd-platform.git
Credentials- rohitrawat025/******(add-global-username with password-Scope: Global jenkin, node, item all child item etc-username: rohitrawat025-password: paste pat tocken ffrom github)
Brabches to build: */Master-Script Path:jenkins/Jenkinsfile

GITHUB UI CONFIGURE:
WEBHOOK CREATION:
Login -Repo: devops-aws-jenkins-docker-cicd-platform-settings-webhooks-Payload URL *: http://15.207.93.132:8080/github-webhook/-Content type *: application/jason-enable ssl verification-just the push event

PAT TOKEN CREATION:
Home-profile settings-developer settings-personal acess token-classic-note: jenkins webhook token-scops: repo and admin:org_hook


Not Secure http://43.205.193.15:81
DevOps Engineer Portfolio
Home Projects
Rohit Rawat
DevOps Engineer | AWS | Docker | Jenkins | Terraform
About Me
DevOps engineer experienced in CI/CD, containerization, AWS infrastructure and automation.
Skills
AWS
Docker
Jenkins
Terraform
Linux
Nginx
Git
Projects
Not Secure http://43.205.193.15:81/projects
My DevOps Projects
Home Projects
Jenkins Master Agent CI/CD
Implemented Jenkins distributed pipeline on AWS EC2 with automated Docker deployment
Technology: Jenkins, Docker, AWS
Github Repository
Spring Boot Docker Deployment
Containerized Java application with Docker and Nginx reverse proxy
Technology: Docker, Nginx, Spring Boot
Github Repository
Terraform AWS Infrastructure
Provisioned VPC, EC2, IAM and networking infrastructure using Terraform
Technology: AWS, Terraform
Github Repository
© 2026 DevOps Portfolio